value, the group ID, and the URL of the bootstrap server (consisting of the host and port):

```
import akka.kafka.ConsumerSettings

object ConsumerPlainSource {
  ...
  val consumerSettings: ConsumerSettings =
      ConsumerSettings(
        config,
        new StringDeserializer(),
        new StringDeserializer())
        .withBootstrapServers("127.0.0.1:9092")
        .withGroupId("group01")
  ...
}
```

By default, along with each value, a Kafka topic stores a key that refers to its partition. This is the default strategy when a Kafka producer commits to Kafka, and it can be changed. And you need to pick a deserializer for both the key and the value. Of course, this depends on which serialization has been previously used when pushing data into the topic. Here we assume the producer has pushed it with UTF-8 encoding. (The default setting for Jackson is UTF-8.)

The `groupId` is used to track consumption of the topic. Consumption is stored in what is called the *offset*, and each `groupId` has its own offset. As soon as a batch of elements from the topic is consumed, a new offset is stored. This marks the last read position in the topic for that `groupId`. This action is called *committing the offset*. By default, the Alpakka plain consumer—`plainSource`—does not commit this offset. When the application restarts, it does not pick up where it left off.